

Learning Plan: Defending the Quadrotor Project

Calibrated to the seven defense questions and the five numbers you must reproduce cold. Designed for a concentrated burst, not a semester.

1. What you are actually learning, and what you are not

The target is not the field. The target is a closed set: the seven questions you already wrote in section 6 of your own record, plus five numbers you should be able to derive on a whiteboard. Once your repository contains a section titled anticipated defense questions, those questions set the floor. Nobody expects you to derive the Kalman gain from the matrix inversion lemma. They do expect you to answer the questions you flagged as expected. That is small and finite.

In scope: the mechanism behind each of your seven answers, and the five derivations in section 4. Out of scope, and fine to skip for your stated angle: the linear-algebra proof of Kalman optimality, the PPO clipped-objective math, and any 6-DOF dynamics. If asked about those, the correct answer is 'I treated that as library-level and focused on the control reasoning,' which is honest and defensible.

2. Which concept each question needs

Defense question	Concept it tests	Tier
PD settles low vs zero error	Steady-state error, force balance	0
Commanded accel in predict	Kalman predict step, phase lag	1
Memoryless net cannot filter	State estimation is temporal	1
Why the network lost	When classical control is optimal	2
Same physics across layers	The plant / regression check	0
2% band, 10 seeds	Settling criterion, noise averaging	0
What is $Q = \sigma_a^2 G G^T$	Process noise, intuition only	1 / 3

3. The tiered plan

Roughly two focused days total. Do the tiers in order. Each ends with a self-test you must pass before moving on.

Tier 0: control basics (about half a day)

- Understand the three PID terms mechanistically, not as a formula. Use contrast table A in section 5.
- Understand why P-only oscillates forever and why D stops it: the equation of motion has no velocity term until D adds one. Derivation in section 4.1.
- Reproduce the steady-state-error result $e = mg/kP$ by hand. Section 4.2.
- Know that the 2% band is the standard settling criterion and that averaging over seeds removes the random sensor-noise draw so numbers repeat.

Tier 1: state estimation (about one day, the heaviest tier)

- Understand the job of a Kalman filter in one sentence: fuse a noisy measurement with a model prediction, weighted by which you trust more right now.

- Understand the predict-update loop at the level of contrast table C, not the matrix algebra.
- Understand what the gain K trades off and that it is recomputed every step.
- Understand the headline fix: feeding commanded acceleration into predict removes the estimate lag that caused a phase lag that destabilized the loop. This is your single most-tested point.
- Understand Q at intuition level only: it encodes how much unmodeled acceleration you expect, which sets how fast the filter is willing to distrust its own model.

Tier 2: the benchmark reasoning (about half a day)

- Understand why PID is near-optimal here: the plant is linear, low-dimensional, and well-modeled, so there is no nonlinearity or coupling for a network to exploit. Contrast table B.
- Understand the fairness argument: a memoryless network cannot filter, so the Kalman filter sits in front of both controllers as a shared perception layer.
- Understand the generalization result: the network learned a fixed offset at 10 m and carries it to unseen setpoints, which is why it degrades at 6 m and 14 m. Section 4.5.

Tier 3: optional depth, skip unless asked

- The matrix form of the Kalman update and why K minimizes the posterior covariance.
- PPO internals: the clipped surrogate objective and the actor-critic split.
- These are library-level for your angle. Learning them is upside, not a requirement.

4. The five numbers you must reproduce cold

Fully worked, with numbers plugged in. If you can rebuild these five on a blank page, you can defend the quantitative claims in the record.

4.1 Why P-only oscillates forever, and why D fixes it

Define the error $e = z_{\text{target}} - z$. With a pure P law the thrust is proportional to e . Write Newton's second law on the vertical axis, cancelling gravity for clarity so only the control term remains. Because z_{target} is constant, the second derivative of e equals minus the second derivative of z .

```
m * z'' = T - mg
let e = z_target - z => z'' = -e''
P-only about the offset: m*(-e'') = kP*e
=> e'' + (kP/m)*e = 0 # undamped oscillator, SHM
solution: e(t) = A*cos(w*t), w = sqrt(kP/m) # never decays
```

There is no first-derivative (velocity) term, so nothing removes energy and the oscillation never dies. The derivative term adds exactly that missing term: its command is kD times de/dt , which introduces a plus $(kD/m)*e'$ into the equation, turning it into a damped oscillator that settles.

```
with D added: e'' + (kD/m)*e' + (kP/m)*e = 0 # damped, decays to zero
```

4.2 Steady-state error of P-only under gravity (no feed-forward)

At equilibrium the drone is motionless, so velocity = 0 and acceleration = 0. Newton then forces thrust to equal weight. With P-only, thrust is kP times the residual error.

```
equilibrium: a = 0 => T = mg
P-only:      T = kP * e_ss
=> kP * e_ss = mg
=> e_ss = mg / kP = (1.0 * 9.81) / 6 = 1.635 m
settle altitude = 10 - 1.635 = 8.365 m ~ 8.37 m
```

This is why the PD config lands near 8.37 m. The offset is forced by physics: without an integral term or a gravity offset, the only way to make hover thrust is to sit at a permanent error.

4.3 Why the derivative on raw noise explodes

The numerical derivative subtracts two consecutive noisy samples and divides by dt . The two noise draws are independent, so their variances add.

```
v_est = (z_t - z_{t-1}) / dt
Var(z_t - z_{t-1}) = sigma_z^2 + sigma_z^2 = 2*sigma_z^2 # independent draws
std = sqrt(2) * sigma_z
noise on v_est = sqrt(2)*sigma_z / dt = 1.414 * 0.25 / 0.01 = 35.4 m/s
```

A 0.25 m sensor wobble becomes a 35 m/s velocity spike every step, then gets multiplied by kD . That is the chatter that hit x232 in the record, and the entire reason the Kalman filter exists in this project.

4.4 The hover offset and the action mapping

The RL environment maps a normalized action to real thrust around a hover offset. Check the three corners.

```
hover = mg = 1.0 * 9.81 = 9.81 N
T = hover + a*(T_max - hover), T_max = 2*mg = 19.62 N
a = 0 -> T = 9.81 N = mg -> net accel 0, holds altitude
a = +1 -> T = 19.62 N = 2*mg -> thrust-to-weight ratio 2
a = -1 -> T = 0 N -> minimum thrust
```

Because $a = 0$ already holds hover, gravity is pre-cancelled, which is precisely why the integral term is unnecessary in the RL environment and only introduces wind-up. This is the answer to why $k_I = 0$.

4.5 Why the network degrades at 6 m specifically

The network carries a fixed steady-state offset of about 0.10 m learned at 10 m. The settling band scales with the setpoint, so compare the offset to the band at each target.

```
band = 0.02 * setpoint
10 m: band = 0.20 m,  offset 0.10 sits well inside  -> settles fine
 6 m: band = 0.12 m,  offset 0.10 sits at the edge  -> noise trips it, 8.5 s
14 m: band = 0.28 m,  offset inside, but extrapolation raises overshoot to 17%
```

5. Concept contrast tables

Table A: the three PID terms

Term	Responds to	Fixes	Failure mode alone
P	current error	gets you moving toward target	oscillates forever, no damping
I	accumulated past error	permanent steady-state offset	winds up, overshoots, lags
D	rate of error change	adds damping, kills oscillation	amplifies sensor noise

Table B: when classical beats learned control

Plant property	Favors PID	Favors learned (RL)
Linearity	linear	strongly nonlinear
Dimensionality	low-DOF (this project)	high-DOF, coupled
Model quality	well-modeled	hard or impossible to model
Example	1-DOF altitude hold	legged locomotion, drone acrobatics

Table C: the two Kalman steps

Step	What it does	Uses
Predict	project the estimate forward with the model	F, the control input u , and Q
Update	correct the prediction with the new measurement	the measurement y , H , R , gain K

6. Self-test: answer these cold, then check

Write each answer on a blank page with no notes, then grade yourself against section 6 of the technical record. If you cannot answer one, that is your next study target, not a reason to move on.

- 1 Why does PD settle below target in the first sim but reach zero error in the RL environment?
- 2 Why must commanded acceleration go into the Kalman predict step, and what breaks without it?
- 3 Why is it not unfair that the network is memoryless?
- 4 Why did the network lose, and when would a network win instead?
- 5 How do you know the physics is identical across all three layers?
- 6 Why a 2% band and why average over 10 seeds?
- 7 What does Q represent, in one sentence, without matrix algebra?
- 8 Reproduce $e_{ss} = mg/k_P$ with numbers, and the $\sqrt{2} \cdot \sigma_z / dt$ noise figure.

7. Minimal resource list

High-yield only. You do not need to finish any of these, only extract the named idea. Budget the times shown.

- **Brian Douglas, control-system videos (YouTube):** the PID intuition and the derivative-noise problem. About 40 minutes. Covers Tier 0.
- **MATLAB Tech Talks, 'Understanding Kalman Filters' series (Melda Ulusoy):** the best short intuition for predict-update and the gain tradeoff, no heavy algebra. About one hour. Covers most of Tier 1.

- **Stable-Baselines3 documentation, PPO page:** read only enough to say PPO is an on-policy actor-critic method that improves a policy by trial and reward. Fifteen minutes. Enough for your angle.
- **Your own record, section 6:** the highest-yield resource. The answers are already written. The work is converting them from text you have to knowledge you hold.

8. The night-before refresher (one page)

Five numbers: $e_{ss} = mg/kP = 9.81/6 = 1.64$ m. Derivative noise = $\sqrt{2} \cdot \sigma_z/dt = 35$ m/s. hover = $mg = 9.81$ N, $T_{max} = 2mg = 19.62$ N, so $a=0$ holds hover. 2% band at 6 m = 0.12 m, which the 0.10 m learned offset sits on. Reward: doing nothing about minus 1000, PPO plateaued about minus 80.

- P oscillates because the equation has no velocity term; D adds it.
- I exists to kill steady-state offset; unnecessary here because the hover offset pre-cancels gravity.
- Kalman fuses noisy measurement with model prediction; gain K is recomputed each step and trades trust.
- The headline fix: commanded acceleration into predict removes estimate lag, hence phase lag, hence instability.
- PID wins because the plant is linear, low-DOF, well-modeled. A network wins on nonlinear high-DOF plants.
- The Kalman filter is a shared front-end so the memoryless network is not unfairly handicapped.

9. How to present it, and one honesty line

Lead with the result, not the setup. Your first sentence in any conversation about this is the result sentence with the numbers in it. A correct negative result, PID beating a network on a plant where classical control should win, reads as judgment. That is a self-promotion asset, so state it plainly rather than burying it as a disappointment.

The one line to hold. You used AI to build above your current level, which is normal and defensible. The defensible claim is 'I used AI as a tool and I understand every decision in this project.' That claim becomes true when you finish this plan. Do not let anyone move you to the adjacent claim 'I built this independently,' which is false and is the only sentence here that could actually harm you. The plan exists to make the true version unassailable.